

Kitchenware object classification

Course Aprendizaje Automático Avanzado

by Álvaro Viguera Capablo, Guillermo Álvarez
González, Samuel Llamas Martínez

3 April 2026



Contents

1. Introduction
2. Proposed solutions
3. Detailed explanation of the selected method
4. Datasets used for evaluating the methods
5. Typical metrics
6. Implementations in Python
7. Conclusions
8. Credits
9. References

1. Introduction

What is kitchenware classification?

- Classification of household kitchen objects from images using Deep Learning
- The aim is to assign a single class label to each input image (“fork”, “plate”, “glass”, ...)
- Consists of using Deep Learning models to recognize object patterns



Why is important kitchenware classification?

- Transitioning from manual sorting to AI-driven automation.
- Kitchenware is among the most common objects in human environments
- There is a high variability in each class (different plate shapes, fork sizes)
- Real-world applications require robust and accurate systems

...Introduction: Main Challenges

Computer Vision Difficulties

- **Severe Occlusion:** Cutlery and dishes are rarely isolated. Objects partially or completely block each other (e.g., a crowded sink)
- **Physical Properties:** Metallic knives and forks reflect light (spectacular reflection), confusing the neuronal network. Glassware adopts background colors and lacks solid borders
- **Visual Variance:**
 - *High intra-class variance:* An espresso cup and a large bowl look completely different
 - *Low inter-class variance:* Spoons and forks share almost identical contours at lower resolutions



2. Proposed solutions - Overview

Typically it is the "state-of-the-art"

Deep Learning Architectures

CNNs



Extract hierarchical spatial features (edges, textures, shapes)



Highly efficient for local pattern recognition in kitchenware.

State of the Art Revision

Deep Learning models for image classification

Vision Transformers (ViTs)



Process images as sequential "patches"



Utilize Self-Attention mechanisms to capture global context and relationships from the first layer.

Method	Year	Description	Evaluation Dataset	Performance (accuracy)	Code	Speed
ResNet-50	2016	Residual connections	ImageNet-1k	76.1%	Yes	High
ViT-B/16	2021	Transformer for images using patches.	ImageNet-1k	77.9%	Yes	Medium
MobileNetV3	2019	Hardware recognition efficiency	ImageNet-1k	75.2%	Yes	Ultra High
EfficientNetV2	2021	Faster training and parameter scaling.	ImageNet-1k	87.3%	Yes	High
Swin-T	2021	shifted windows.	ImageNet-1k	81.3%	Yes	Medium
DenseNet-121	2017	Dense blocks connecting every layer.	ImageNet-1k	75.0%	Yes	Medium
ConvNeXt-B	2022	CNN with Transformer techniques.	ImageNet-1k	83.8%	Yes	High
DeiT-S	2021	teacher-student distillation.	ImageNet-1k	79.8%	Yes	High
MaxViT	2022	Multi-axis attention	ImageNet-1k	86.5%	Yes	Low

3. Detailed Explanation: Vision Transformer (ViT-B/16)

Pre-processing (The Setup):

- Instead of looking at pixels, the model chops the image into small 16x16 squares (patches).
- It treats these image patches just like words in a sentence.

Descriptors (How it extracts features):

- No traditional convolutions. It uses "*Self-Attention*".
- This means the model looks at the *whole* image at once to see how different parts (e.g., the handle and the head of a spoon) relate to each other.

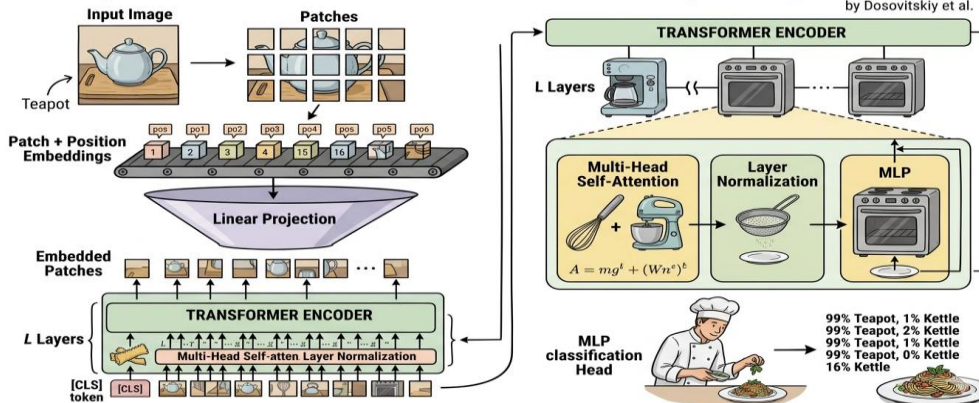
Classification (Making the decision):

- It adds a special summary token that travels through the network gathering all the clues.
- At the end, a simple decision layer reads this summary and predicts the correct kitchenware class.

Metrics: * Solid performance: **77.9% accuracy** on the ImageNet-1k dataset.

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

by Dosovitskiy et al.



3.Detailed Explanation: EfficientNetV2

Pre-processing (The Setup):

- Uses **"Progressive Resizing"**: Starts training with tiny images and slowly increases their size. This saves a lot of time!
- Applies data augmentation (mixing and twisting images) so the model doesn't memorize them.

Descriptors (How it extracts features):

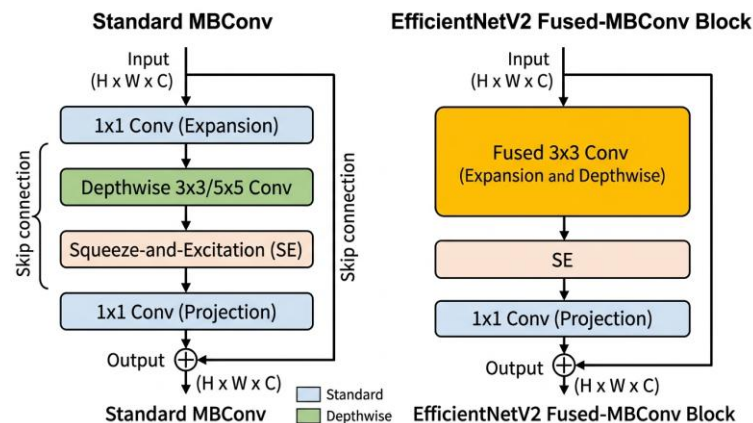
- Uses smart, highly optimized convolutional blocks (Fused-MBConv).
- It works step-by-step: early layers find simple edges, and deeper layers recognize complex shapes like a fork's prongs.

Classification (Making the decision):

- It "squashes" all the visual features it found into a single list of numbers (Global Average Pooling).
- A final layer turns those numbers into class probabilities (e.g., 90% plate, 10% bowl).

Metrics:

- Excellent balance of speed and power: **87.3% accuracy** on ImageNet-1k.



3.Detailed Explanation: ResNet-50

Pre-processing (The Setup):

- Images are resized to a standard 224x224 pixels.
- Colors are normalized, and we use basic tricks like horizontally flipping the images to give the model more variety.

Descriptors (How it extracts features):

- Uses classic Convolutional Neural Networks (CNNs), but with a twist: *Skip Connections*.
- These act as "shortcuts" that allow information to flow easily through 50 layers without getting lost or degraded.

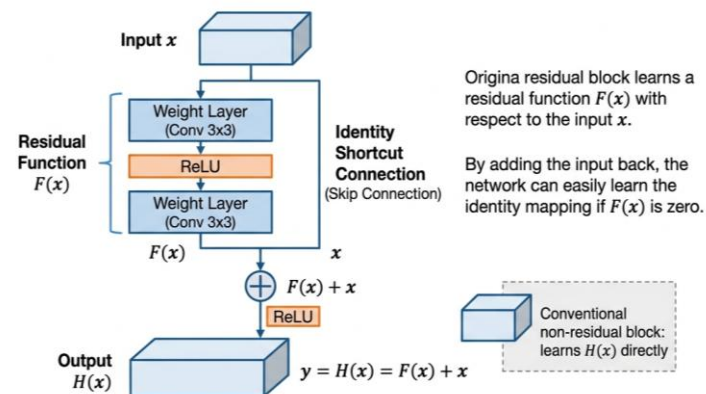
Classification (Making the decision):

- After passing through the 50 layers of feature extraction, the data is averaged out.
- A final *Softmax* layer calculates the mathematical probability for each object (knife, spoon, etc.).

Metrics:

- Reliable and heavily tested: **76.1% accuracy** based on the Group-1k evaluation.

RESIDUAL BLOCK ARCHITECTURE (HE ET AL., 2015)



4. Datasets for Kitchenware Classification

- **Kitchenware Classification (Kaggle)**
 - **Date:** 2022
 - **Link:** www.kaggle.com/competitions/kitchenware-classification
 - **Composition:** >5,500 images. 6 classes (cups, glasses, plates, spoons, forks, knives).
- **Knifey-Spoony Dataset**
 - **Date:** 2018
 - **Link:** github.com/Hvass-Labs/knifey-spoony
 - **Composition:** 4,700 images (200x200px). 3 classes (knives, spoons, forks) on varied backgrounds.
- **Edinburgh Kitchen Utensil Database (EKUD)**
 - **Date:** 2010s
 - **Link:** homepages.inf.ed.ac.uk/rbf/UTENSILS/
 - **Composition:** 897 images. 20 classes of specific kitchen utensils (soup spoons, spatulas, bread knives...).
- **Alicante Kitchenware Dataset (AKDRM)**
 - **Date:** 2022
 - **Link:** dlsi.ua.es/~jgallego/datasets/utensils/
 - **Composition:** 1,480 images. 12 classes with real-world backgrounds and varied lighting.



4. Datasets for Kitchenware Classification

MS COCO (Common Objects in Context)

- **Date:** 2014
- **Link:** cocodataset.org
- **Composition:** >330,000 images total. Contains specific relevant classes evaluated in everyday contexts: fork, knife, spoon, bowl, cup.

ImageNet (ILSVRC)

- **Date:** 2012
- **Link:** image-net.org
- **Composition:** >14 million images total. Includes heavily benchmarked synsets for kitchenware (spoon, fork, knife, plate).

Open Images V7 (Google)

- **Date:** 2022 (V7 release)
- **Link:** storage.googleapis.com/openimages/web/index.html
- **Composition:** >9 million images total. Highly complex scenes containing annotated classes like Spoon, Fork, Knife, Plate, Mug.

Our Group Dataset

- **Date:** 2026
- **Link:** [Internal agora upload](#)
- **Composition:** >648 images total. 6 classes: cups, forks, glasses, knives, plates, spoons

4. Datasets for Kitchenware Classification

Dataset Name	Date	Link	Composition
Kitchenware Classification (Kaggle)	2022	www.kaggle.com/competitions/kitchenware-classification	>330,000 images total. Contains specific relevant classes evaluated in everyday contexts: fork, knife, spoon, bowl, cup.
Knifey-Spoony Dataset	2018	github.com/Hvass-Labs/knifey-spoony	Medium
Edinburgh Kitchen Utensil Database (EKUD)	2010s	homepages.inf.ed.ac.uk/rbf/UTENSILS/	Ultra High
Alicante Kitchenware Dataset (AKDRM)	2022	dlsi.ua.es/~jgallego/datasets/utensils/	High
Our Group Dataset	2026	Internal agora upload	Medium
MS COCO	2014	cocodataset.org	Medium
ImageNet (ILSVRC)	2012	image-net.org	Medium
Open Images V7 (Google)	2022	storage.googleapis.com/openimages/web/index.html	High

5. Typical Metrics Used in Image Classification

Accuracy:

What it is: The percentage of total correct predictions out of all predictions made.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

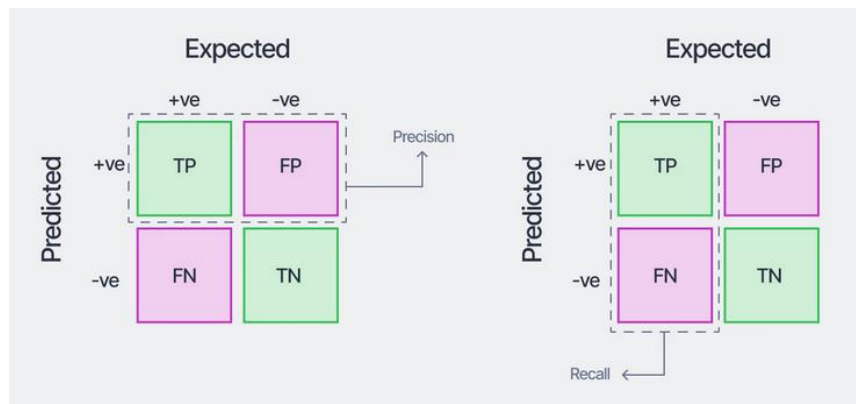
What it is: The ratio of correctly predicted positive observations to the total predicted positive observations.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall:

What it is: The ratio of correctly predicted positive observations to the all observations in actual class.

$$\text{Recall} = \frac{TP}{TP + FN}$$



5. Typical Metrics (Imbalanced Data & Top-K)

F1-Score:

What it is: The harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Top-1 vs. Top-5 Accuracy:

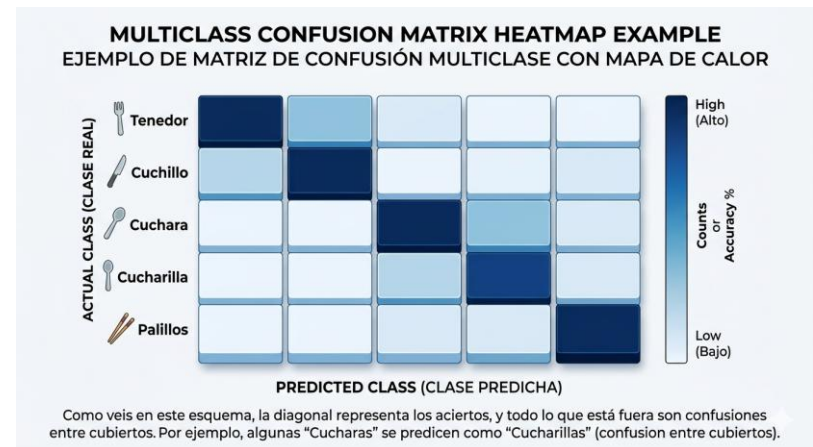
Top-1: The model's highest probability prediction must be exactly the correct answer.

Top-5: The correct answer must be among the model's top 5 highest probability predictions. (Standard in ImageNet evaluations for ResNet/ViT).

Confusion Matrix:

What it is: A table that visualizes the performance of the classification model.

Why use it: It instantly shows us which classes are getting confused by the network (e.g., distinguishing between a spoon and a fork might be harder than a knife and a plate).



6. Implementations in Python: ViT/B16

Implementation 1: HuggingFace transformers

Link: huggingface.co/docs/transformers/model_doc/vit

- **Release Date:** October 2020
- **Advantages:** Extremely user-friendly API; includes built-in ViTImageProcessor for exact paper-replication preprocessing.
- **Limitations:** High level of abstraction makes modifying internal attention layers complex.

Implementation 2: PyTorch torchvision.models

Link: pytorch.org/vision/main/models/vision_transformer.html

- **Release Date:** 2022 (Added to Torchvision)
- **Advantages:** Native PyTorch integration; lightweight and easy to insert into standard custom training loops.
- **Limitations:** Requires writing manual preprocessing pipelines from scratch.

Main Differences: HuggingFace is optimal for fast deployment and unified pipelines, while PyTorch native offers better low-level control for researchers.

```
# Hugging Face
from transformers import ViTForImageClassification
model = ViTForImageClassification.from_pretrained('google/vit-base-patch16-224')

# PyTorch Torchvision
import torchvision.models as models
model = models.vit_b_16(weights='DEFAULT')
```

6. Implementations in Python: EfficientNetV2

Implementation 1: Official Google AutoML (TensorFlow)

- *Link:* github.com/google/automl/tree/master/efficientnetv2
- *Release Date:* April 2021
- *Advantages:* Original implementation by the authors; guarantees exact replication of the paper's results.
- *Limitations:* Written in TensorFlow 1.x/2.x, which can be harder to integrate if the group uses PyTorch.

Implementation 2: PyTorch Image Models (timm)

- *Link:* github.com/huggingface/pytorch-image-models
- *Release Date:* 2021 (EfficientNetV2 added)
- *Advantages:* The absolute industry standard for PyTorch; provides highly optimized state-of-the-art pre-trained weights.
- *Limitations:* Requires managing an external third-party dependency (pip install timm).
- **Main Differences:** The official Google repository is authentic but rigid (TensorFlow), whereas timm provides massive flexibility and modern PyTorch compatibility.

```
# Implementation 1: Official Google (TensorFlow)
import tensorflow as tf
model = tf.keras.applications.EfficientNetV2B0(weights='imagenet')

# Implementation 2: PyTorch Image Models (timm)
import timm
model = timm.create_model('efficientnetv2_rw_m', pretrained=True)
```


6. Implementations in Python: ResNet-50

Implementation 1: PyTorch torchvision.models

- *Link:* github.com/pytorch/vision/blob/main/torchvision/models/resnet.py
- *Release Date:* 2017
- *Advantages:* Native framework integration, highly stable, one-line loading for pre-trained ImageNet weights (weights='DEFAULT').
- *Limitations:* Barebones implementation; lacks some modern experimental architectural tweaks.

Implementation 2: Keras / TensorFlow Applications

Link: keras.io/api/applications/resnet/

- *Release Date:* 2017
- *Advantages:* The Keras API is extremely beginner-friendly and great for fast prototyping with minimal code.
- *Limitations:* The TensorFlow/Keras ecosystem can be bulkier and slower for custom debugging compared to PyTorch

Main Differences: PyTorch provides a more "Pythonic", low-level debugging experience, while Keras focuses on high-level speed and simplicity for immediate deployment.

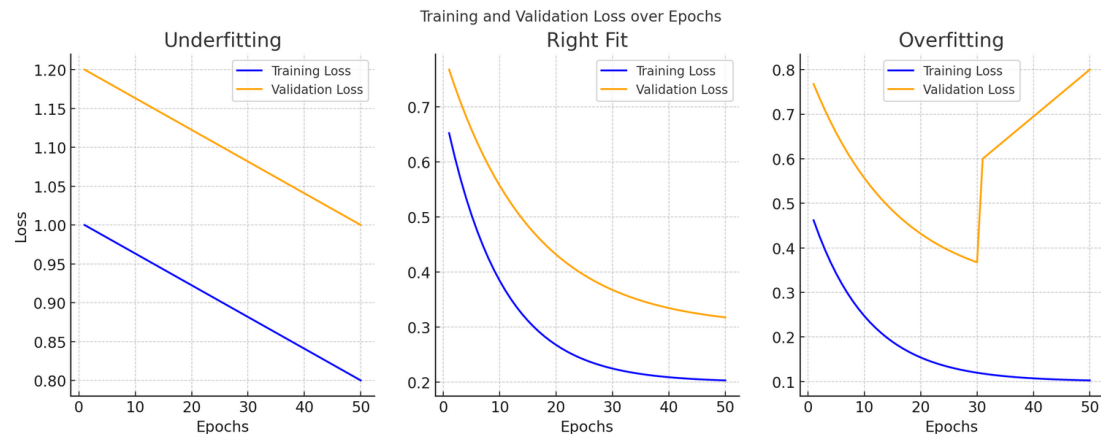
```
# Implementation 1: PyTorch Torchvision
import torchvision.models as models
model = models.resnet50(weights='DEFAULT')
```

```
# Implementation 2: Keras / TensorFlow Applications
from tensorflow.keras.applications import ResNet50
model = ResNet50(weights='imagenet')
```

7. Regularization Strategies

Goal: Prevent overfitting, especially in the small Group Dataset

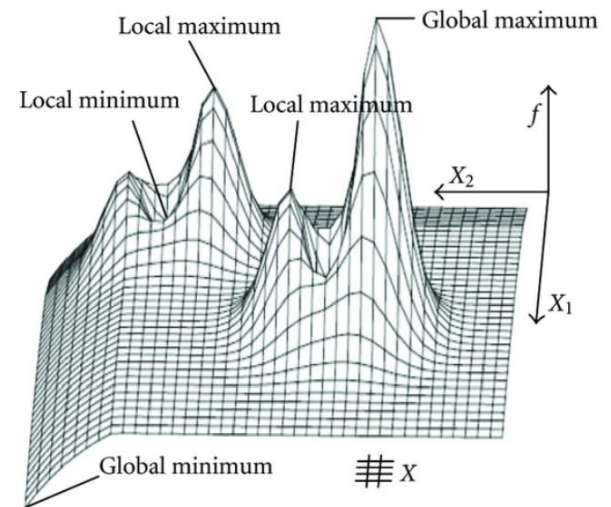
- **Small Datasets:** for our group dataset, we would use "*Data Argumentation*": flips, rotations, and color changes in order to multiply our 648 images.
 - **High Dropout (0.5 - 0.7):** To force the network to find multiple paths for the same result.
 - **L2 Weight Decay:** Penalizing large weights to keep the model simple.
- **Large Datasets:** for pooled/public datasets.
 - **Lower Dropout (0.2 - 0.3):** Since the data volume already acts as a natural regularizer.
- **Advanced techniques**
 - **Mixup & CutMix:** Creating "hybrid" images (e.g., half spoon, half fork) to smooth decision boundaries.
 - **Early Stopping:** Monitoring Validation Loss to stop training before the model starts "memorizing" backgrounds.



7. Optimization Strategies

Goal: Efficiently navigate the error landscape to find the global minimum.

- **Small Datasets:**
 - **SGD with Momentum (0.9):** Slower but more stable. It prevents the model from "jumping" too much when it sees an outlier in a small batch.
 - **L2 Weight Decay:** Penalizing large weights to keep the model simple.
- **Large Datasets:** for pooled/public datasets.
 - **AdamW:** A modern version of Adam that handles Weight Decay much better, leading to faster convergence in large-scale datasets.
- **Advanced techniques**
 - **Cosine Annealing :** A scheduler that lowers the learning rate following a cosine curve, allowing for a "soft landing" at peak accuracy.
 - **One-Cycle Policy:** Starting with a low LR, peaking in the middle, and ending very low to find the flattest (most robust) minimums.



7. Loss Function Strategies

Goal: Mathematically define "error" to handle class imbalance and overconfidence.

- **Small Datasets: Imbalance**

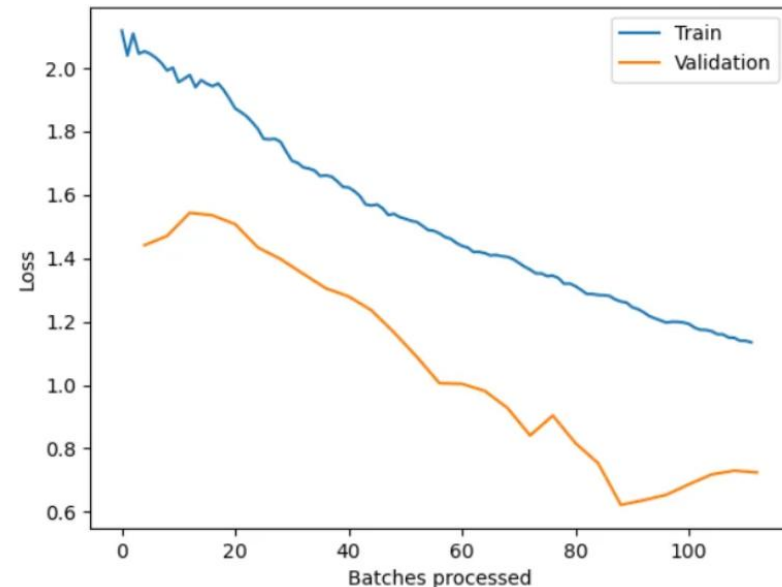
- **The Problem:** If we have 200 plates but only 30 spoons, the model will ignore the spoons.
- **Focal Loss:** It down-weights "easy" examples (plates) and focuses the gradient on "hard" examples (the few spoons we have).

- **Large Datasets: Overconfidence**

- **Label Smoothing:** Prevents the model from being 100% sure of its prediction (e.g., target 0.9 instead of 1.0). This makes the model more robust to noisy labels in public datasets.

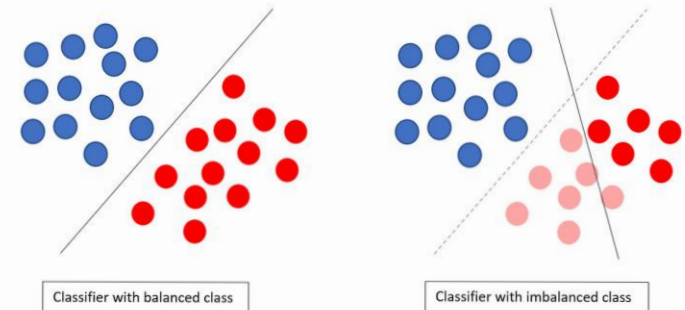
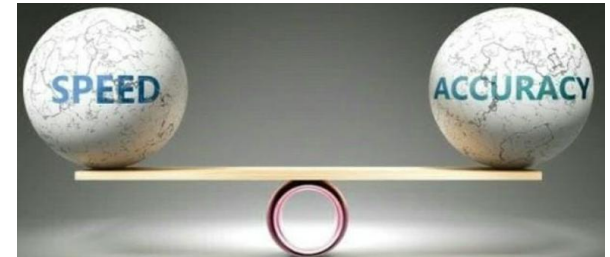
- **Advanced techniques**

- **Class-Balanced Loss:** Scaling the loss by the "effective number" of samples in each class.
- **Knowledge Distillation:** Using a large "Teacher" model (like a huge ViT) to provide "soft labels" to our smaller "Student" model.



8. Conclusions

- **Best Model: EfficientNetV2**
 - It offers the highest balance between Accuracy and computational efficiency (speed / memory)
- **The main drawback is: Class Imbalance**
 - Our small dataset makes it difficult for models to generalize without biased predictions
- **The performance decreases when** background noise is high or when lighting conditions differ significantly from the training set.
- **Using ViT-B/16 is better than ResNet-50 in large-scale scenarios** where global object relationships (attention) are more important than simple local textures
- **Using MobileNetV3 is better than MaxViT in edge devices** where low latency and low power consumption are the priority over raw precision.
- **The implementation evaluated requires a GPU for** efficient training and fine-tuning, as the high number of parameters in modern architectures makes CPU-only training unfeasible.



8. Credits

- **Alvaro Viguera:**
 - Investigation (ViT-B/16): Researched Vision Transformer architectures.
 - Writing - Original Draft: Prepared the initial manuscript.
 - Resources: Responsible for collecting datasets, organizing essential study materials and Loss Function Strategies
- **Samuel Llamas:**
 - Investigation (EfficientNetV2): Researched optimization and efficiency methods.
 - Writing - Original Draft: Contributed to the research and drafting process
 - Datasets for Kitchenware Classification, typical metrics used in image classification and Optimization Strategies
- **Guillermo Álvarez:**
 - Investigation (ResNet-50): Researched classical CNN architectures.
 - Visualization: Prepared all the figures, diagrams, summary tables and Regularization Strategies
 - Supervision: Provided oversight, and final review of the research activities.



9. References

- 1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), pp. 770-778. (El paper original de ResNet).
- [2] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929. (El paper original de ViT).
- [3] Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. In International Conference on Machine Learning (ICML), PMLR, pp. 10096-10106. (El paper de EfficientNetV2).
- [4] Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101. (La referencia para AdamW, que mencionaste en Optimización).
- [5] Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In Proceedings of the IEEE international conference on computer vision (ICCV), pp. 2980-2988. (La referencia para Focal Loss).
- [6] Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2017). mixup: Beyond empirical risk minimization. arXiv preprint arXiv:1710.09412. (Referencia para Mixup, técnica avanzada de regularización).



Thank you for your attention

